

FEDERAL UNIVERSITY
OF SANTA CATARINA

Laboratório 6:
Síntese de Máquina de Estados

EEL5105 – Circuitos e Técnicas Digitais

Introdução

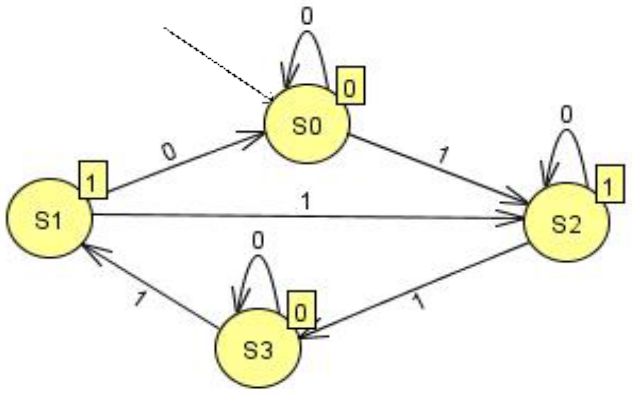
Tarefas

Introdução

- **Objetivos**
  - Entender o conceito de máquinas de estados (FSM).
  - Entender o conceito de circuito sequencial controlando o fluxo de atividades de circuito combinacional.
  - Entender o processo de síntese de FSMs em VHDL.
  - Entender o funcionamento de contadores.
  - Estudo de caso: projeto e implementação em VHDL de um contador baseado em máquinas de estados.

Introdução

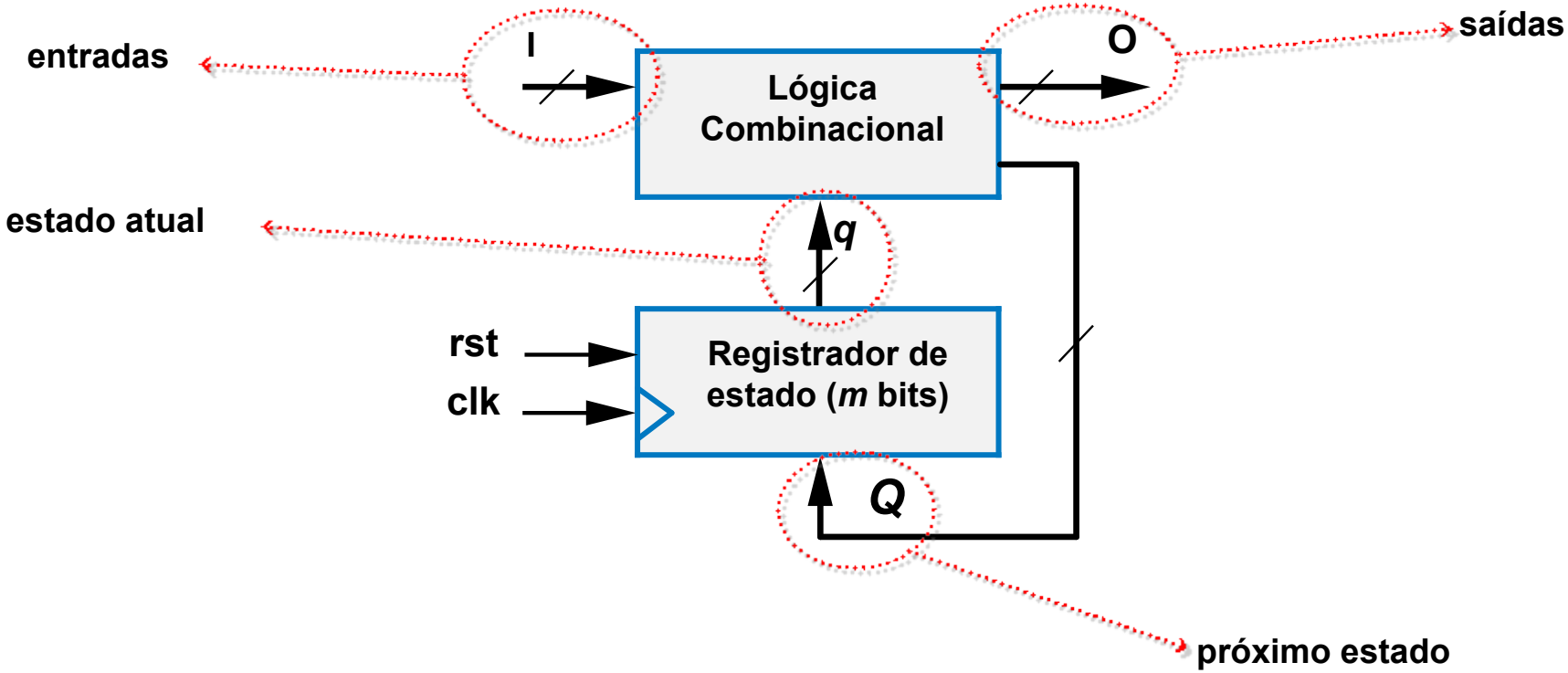
- Modelo de comportamento composto por:
  - Estados
  - Transições
  - Ações



- **Estado**
  - Armazena informação sobre o passado refletindo as modificações das entradas do início até o presente momento
- **Transição**
  - Indica uma troca de estado e pode envolver condição para habilitar a mudança de estado
- **Ação**
  - Descrição da atividade que deve ser executada em um determinado instante

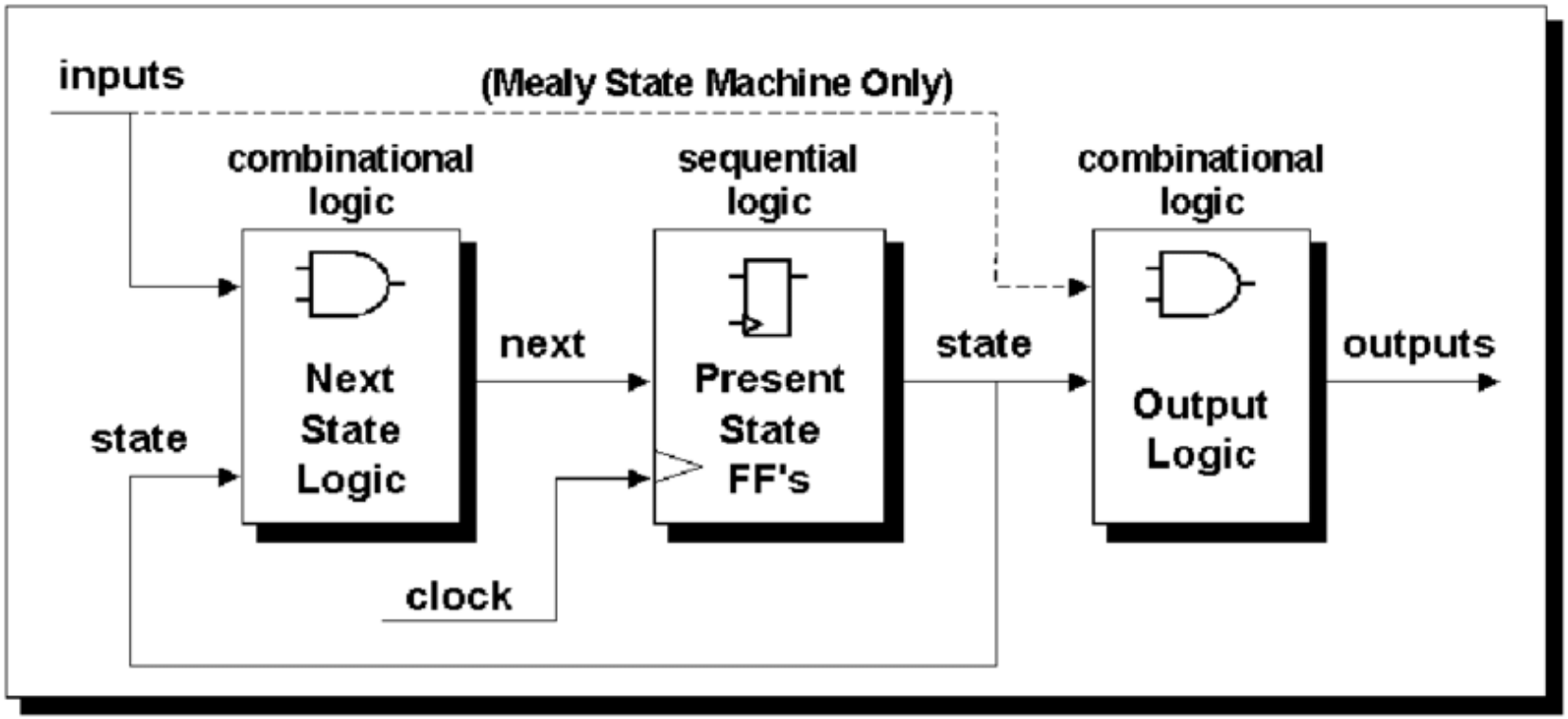
Introdução

- **Módulos:**
  - Registrador para armazenar o “estado atual”;
  - Circuitos lógicos que geram “saída” e o “próximo estado”.



Introdução

- Registrador para armazenar o “estado atual”;
- Circuitos lógicos que geram “saída” e o “próximo estado”.
- **Observar que todas as saídas de estado são simultaneamente atualizadas na borda de clock.**



Source: Cummings, Clifford. (2002). The Fundamentals of Efficient Synthesizable Finite State Machine Design using NC-Verilog and BuildGates.

Introdução

Máquinas de Estado:

- Geram saídas (que podem ser vazias) para cada estado.

Moore:

- transição de estados depende do estado atual e do tempo (clock)

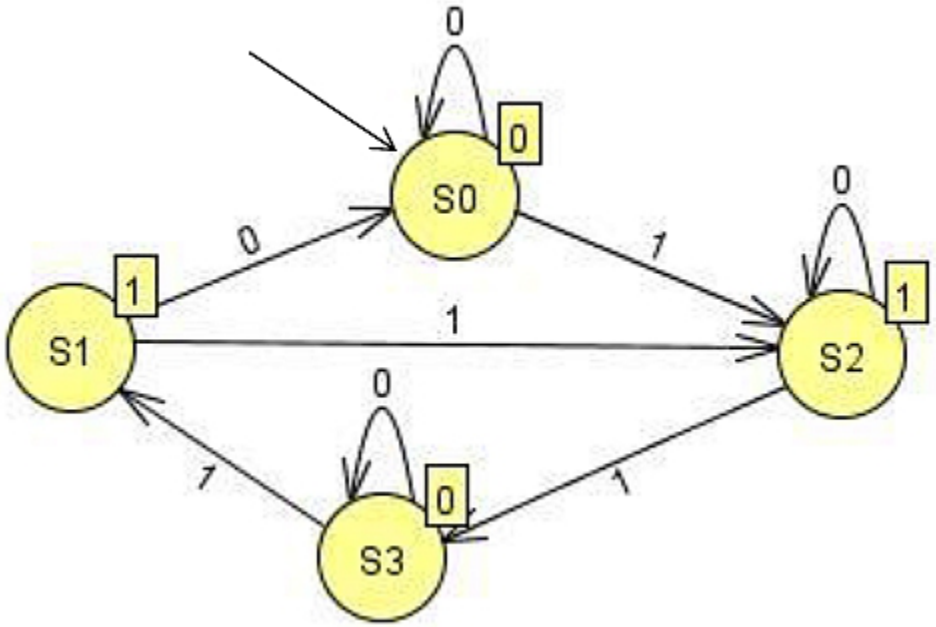
Mealy:

- transição de estados depende do estado atual, do tempo e de valor(es) de entrada.

Introdução

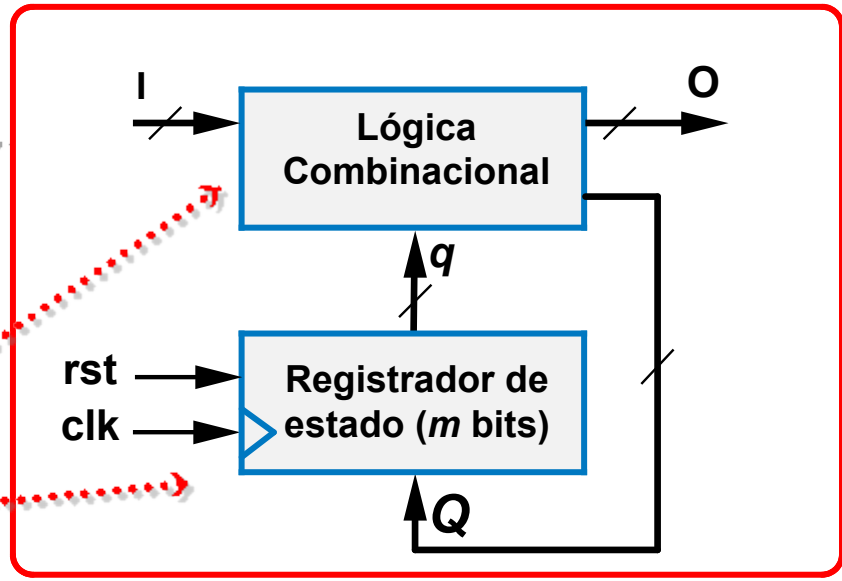
- **Armazenamento do "estado atual"**
  - Registrador construído a partir de flip-flops.
  - **Observar que todas as saídas de estado são simultaneamente atualizadas na borda de clock (máquina de estados síncrona)**
- **Geração do "próximo estado"**
  - Circuito combinacional; ou
  - Tabela verdade da lógica de próximo estado armazenada em uma memória (ROM, Flash, RAM, ...)
- **Geração da "saída"**
  - Circuitos lógicos que geram saídas a partir do estado atual.

Introdução




```
entity Mealy is
port(          clock, reset, X: in std_logic;
               Z: out std_logic    );
end Mealy;
architecture bhv of Mealy is
               type STATES is (S0, S1, S2, S3);
               signal EA, PE: STATES;
begin
               P1: process(clock, reset)
               begin
                     if reset= '1' then  EA  <= S0;   elsif clock'event and clock= '1' then EA <= PE;  end if;
               end process;
               P2: process(EA,X)
               begin
                    case EA is
                          when S0 =>            Z <= '0';
                                                if X  = '0' then  PE  <= S0; else PE <= S2;  end if ;
                          when S1 =>            Z <= '1';
                                                if X  = '0' then  PE  <= S0; else PE <= S2;  end if ;
                          when S2 =>            Z <= '1';
                                                if X  = '0' then  PE  <= S2; else PE <= S3;  end if ;
                          when S3 =>            Z <= '0';
                                                if X  = '0' then  PE  <= S3; else PE <= S1;  end if ;
                    end case;
               end process;
end bhv;
```

Introdução




```
entity Mealy is
port(          clock, reset, X: in std_logic;
               Z: out std_logic    );
end Mealy;
architecture bhv of Mealy is
               type STATES is (S0, S1, S2, S3);
               signal EA, PE: STATES;
begin
               P1: process(clock, reset)
               begin
                     if reset= '1' then  EA  <= S0;   elsif clock'event and clock= '1' then EA <= PE;  end if;
               end process;
               P2: process(EA,X)
               begin
                    case EA is
                          when S0 =>          Z <= '0';
                                              if X  = '0' then  PE  <= S0; else PE <= S2;  end if ;
                          when S1 =>           Z <= '1';
                                              if X  = '0' then  PE  <= S0; else PE <= S2;  end if ;
                          when S2 =>          Z <= '1';
                                              if X  = '0' then  PE  <= S2; else PE <= S3;  end if ;
                          when S3 =>          Z <= '0';
                                              if X  = '0' then  PE  <= S3; else PE <= S1;  end if ;
                    end case;
               end process;
end bhv;
```

Introdução

Tarefas

Tarefas

- Implementar uma **FSM** em VHDL a partir da metodologia teórica para geração de uma sequência cíclica com uma saída de 4 bits.
- FSM com ***reset*** assíncrono, usando o botão **KEY(0)** para reinicializar a sequência.
- A tarefa vai estar dividida em duas partes:
  - **Tarefa 1:** Implementação **e simulação** de **FSM** usando abordagem de implementação de máquina de estados com dois ***process***.
  - **Tarefa 2:** Implementação **e emulação** da **contador.**

Tarefa 1: Implementação e simulação da FSM

Os 3 passos para projetar uma FSM em VHDL:

- **Passo 1:** Descrição funcional do sistema (requisitos);
- **Passo 2:** Representação gráfica da FSM (diagrama de estados);
- **Passo 3:** Criar projeto e descrever o comportamento da FSM a partir do diagrama de estados em VHDL;
- **Passo 4**: Teste e verificação.

Tarefa 1: Implementação e simulação da FSM

- **Passo 1: Descrição funcional do sistema:**

  - No estado inicial, a FSM deve ter **o último número da matricula do aluno**. Em caso de dupla, utilizar o maior valor.
  - Sequências (**apenas fazer a iniciada pelo último número de matricula**):

0 a 1 a 2 a 3 a 4 a 0...
1 a 2 a 3 a 4 a 5 a 1...
2 a 3 a 4 a 5 a 6 a 2...
3 a 4 a 5 a 6 a 7 a 3...
4 a 5 a 6 a 7 a 8 a 4...
5 a 6 a 7 a 8 a 9 a 5...
6 a 7 a 8 a 9 a A a 6...
7 a 8 a 9 a A a b a 7...
8 a 9 a A a b a C a 8...
9 a A a b a C a d a 9...

Tarefa 1: Implementação e simulação da FSM

- **Passo 1: Descrição funcional do sistema:**

  - **IMPORTANTE!** **Como exemplo, apresenta-se a implementação de sequência para aluno que tenha '1' como último número de matricula; assim, o estado inicial será "0001" (ou 1 em decimal). Metodologia similar deve ser utilizada para implementar as demais sequências.**

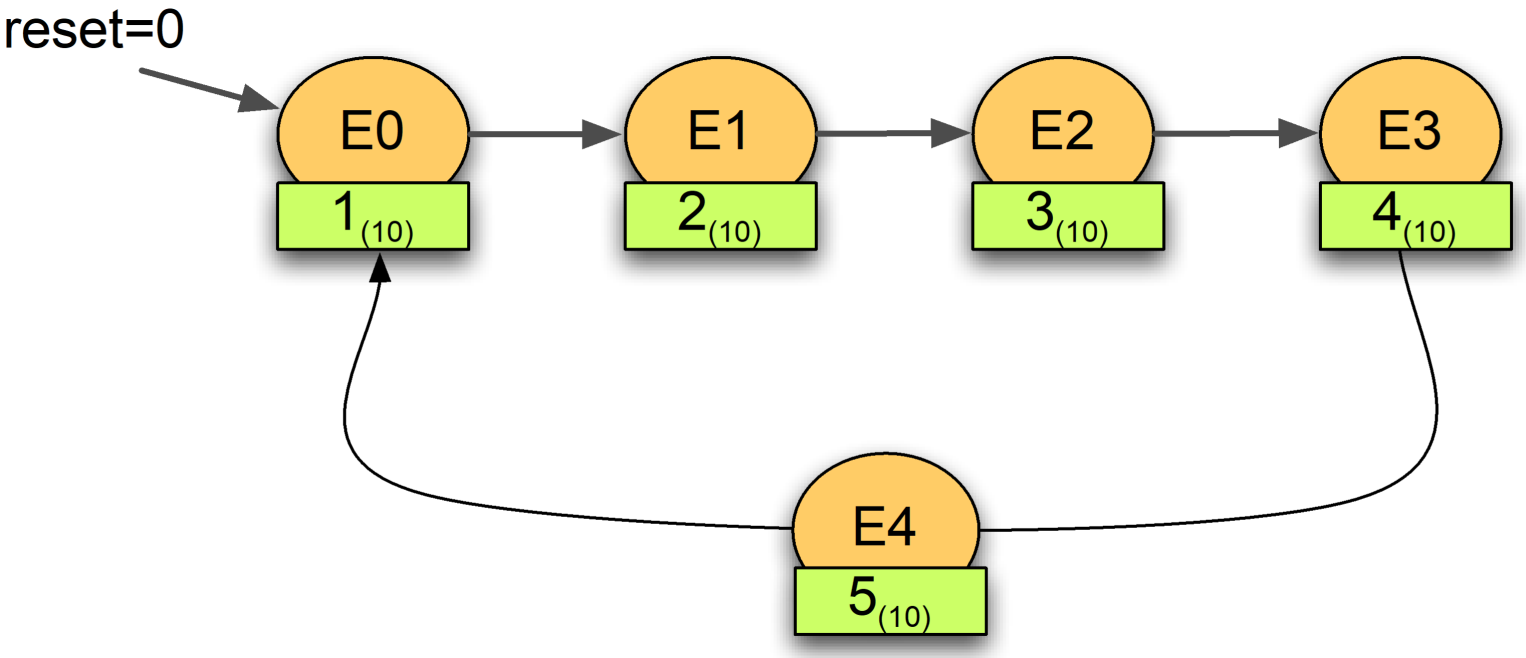
  - A transição de 1 para 0 no relógio, causará o incremento do valor de saída.

  - O último estado da FSM fornecerá a saída com o caractere "5" em decimal. Após o último estado, a FSM deve retornar ao primeiro estado (apresentar saída "1").

  - A qualquer instante, quando o botão de RESET for pressionado, a FSM também deverá retornar para o estado inicial, apresentando a saída "1".

Tarefa 1: Implementação e simulação da FSM

- **Passo 2: Representação gráfica da FSM.**



Inputs:
•Reset
•Clock

Output:
•Contagem

Tarefa 1: Implementação e simulação da FSM

- **Passo 3: Criar projeto no Quartus II e descrever, em VHDL, a FSM a partir do diagrama de estados.**

```
library ieee;
use ieee.std_logic_1164.all;
  entity FSM_Conta is port(
             clock: in std_logic;
             reset: in std_logic;
             contagem: out std_logic_vector(3 downto 0)
             );
  end FSM_Conta;
  architecture bhv of FSM_Conta is
      type STATES is (E0, !!!!!completar com estados );
      signal EA, PE: STATES;
  begin

      P1: process(clock, reset)
      begin
                if reset= '1' then
                            EA  <= E0;
                   elsif clock'event and clock= '0' then
                            EA <= PE;
                end if;
      end process;

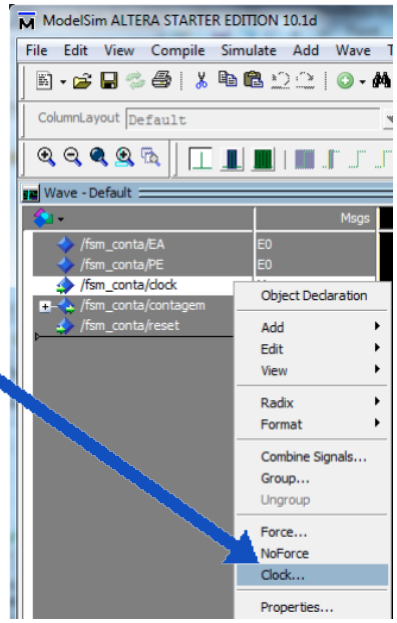
      P2: process(EA)
      begin
             case EA is
                 when E0 =>
                                  contagem <= "0001";
                                  PE <= E1;
                 when ???? =>
                                  contagem <= ????;
                                  PE <= ????;
                 when ???? =>
                                  ???? ;
               end case;
      end process;
  end bhv;
```

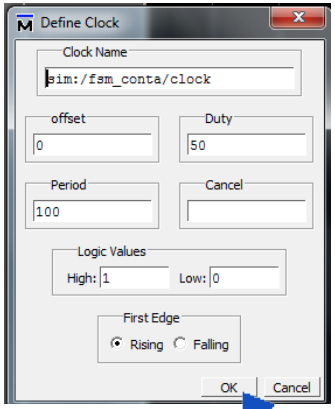
Tarefa 1: Implementação e simulação da FSM

- **Passo 3**: **Compile** o projeto **FSM_conta** e faça um **análise dos erros** obtidos.

- **Passo 4: Teste e verificação.**
- Uma vez implementado e compilado o VHDL da FSM, abrir o *Modelsim*.

Para o relógio selecionar *Clock*



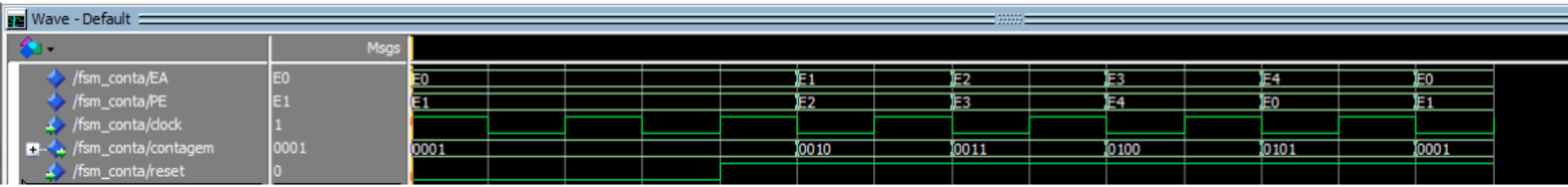


Pressionar OK

Tarefa 1: Implementação e simulação da FSM

- **Passo 4:** **Teste e verificação.**
- Uma vez implementado e compilado o VHDL da FSM, abrir o *Modelsim*.
- Uma vez todas as entradas e saídas na tela *wave* selecione ***clock*** (com os valores padrão) usando o botão direito tal e como foi explicado no Lab5.

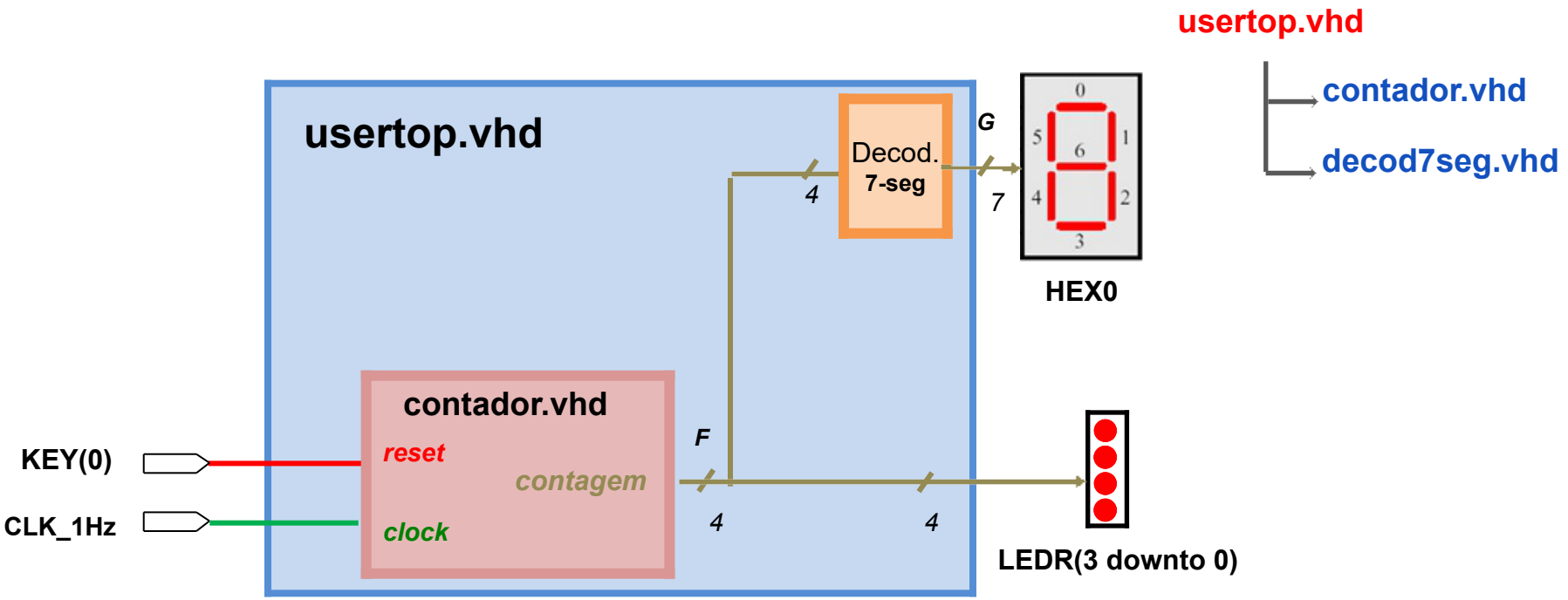
Resultados esperados para sequencia com inicio em "0001"



- **Passo 5:** Visualizar o diagrama de estados gerado pelo **Quartus** em **Tools-> Netlist Viewers-> State Machine Viewer.**

Tarefa 2: Implementação do contador

- **Passo 6:** Implementação **e emulação** do **contador**.
- Devido ao fato de que as sequências da Tarefa 1 são contagens cíclicas, podemos obtê-las-las usando contadores.
- Nessa segunda tarefa, o aluno apenas terá de fazer a emulação e precisará do decodificador para poder visualizar a contagem no HEX0 como é mostrado no diagrama de blocos a ser implementado.



Tarefa 2: Implementação de contador

- **Passo 6:** Exemplo de VHDL de um contador de 5 bits:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity contador is
      port (
                  reset:      in std_logic;
                  clock:      in std_logic;
                  contagem:   out std_logic_vector(4 downto 0)
          );
end contador;
architecture circuito of contador is
signal conta: std_logic_vector(4 downto 0);
begin
P1: process(clock, reset)

begin
         if reset= '0' then conta <= "00000";
         elsif clock'event and clock= '1' then

                   if conta = "11111" then conta <= "00000";
                   else conta <= conta + 1;
                   end if;

               end if;
end process;
contagem <= conta;
end circuito;
```

IMPORTANTE! O aluno deverá modificar o contador para obter a sequência desejada de 4 bits

Tarefa 2: Implementação de contador

Passo 6: A *entity* de **usertop.vhd** deve ser a seguinte:

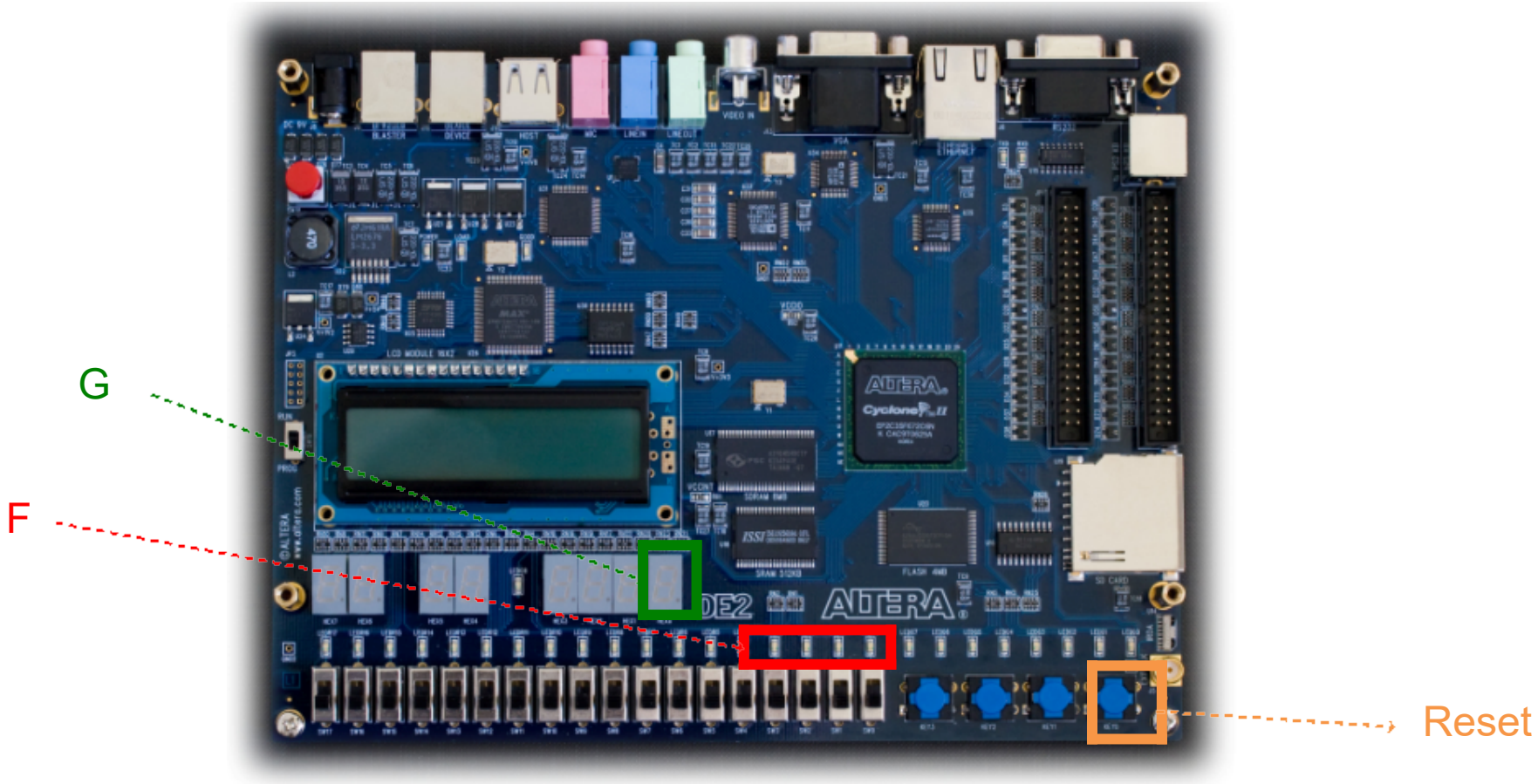
```
entity usertop is
port(
        CLK_1Hz: in std_logic;
        KEY:       in std_logic_vector(1 downto 0);  -- apenas para referenciar KEY(0)
        LEDR:     out std_logic_vector(3 downto 0);
        HEX0:     out std_logic_vector(6 downto 0)
    );
end usertop;

architecture circuito of usertop is


...
end circuito;
```

Tarefa 2: Implementação com relógio de 1Hz

- **Passo 6:** Aqui é apresentada a Interface com a placa DE2 de usertop.vhd. **Lembre-se de que** os push buttons (KEY), **quando pressionados**, fornecem '0'**. Dica:** Modificar seção *arquitecture* no arquivo contador.vhd



Tarefa 2: Implementação com relógio de 1Hz

Passo 7: Fazer a emulação na placa DE2 do circuito **usertop** usando o **emulador on-line**.

(Emulador on-line): Com as devidas adaptações feitas nos arquivos abra o emulador https://fpgaemu.sites.ufsc.br/login e faça ***UPLOAD*** dos arquivos VHDL que deseja utilizar na emulação e compile. Em seguida, inicie a emulação e verifique o funcionamento.